

מכונה תקשוב · יחידה 4

עיצוב משתנים וקוד נקי

קוד שעובד זה לא מספיק — היום לומדים לכתוב קוד שגם בני אדם יכולים לקרוא ולהבין.

תזכורת מהיחידה הקודמת

מה למדנו ביחידה 3?

- **אופרטורים** מבצעים חישובים: `+` `-` `*` `/` `//` `%` `**`
- **ביטוי** הוא כל קטע קוד ש Python-מחשבת לכדי ערך יחיד
- ל Python יש **סדר עדיפויות** קבוע, וסוגריים תמיד ברמה הכי גבוהה
- אפשר לשלב משתנים בתוך ביטויים, ולשמור את התוצאה במשתנה חדש

טיפ. עד עכשיו קראנו למשתנים שלנו `x`, `b`, `a`. היום נלמד למה זה בעצם לא הרגל טוב.

מטרות השיעור

מה נלמד היום?

- למה **שם משתנה טוב** הוא חלק אמיתי מכתובת קוד, לא "קישוט"
- מה **מותר** ומה **אסור** בשם משתנה בפייתון
- מוסכמת `snake_case` — איך פייתון "אוהבת" לקרוא למשתנים
- **מילים שמורות** (reserved words) — השמות שאסור "לגנוב" מפייתון
- איך כותבים **הערות** (comments) שבאמת עוזרות למי שקורא את הקוד

לפני שמתחילים

קוד הוא מכתב לבני אדם

המחשב לא באמת אכפת לו איך קוראים למשתנים — עובד בדיוק כמו `total_price`. אבל **בני אדם** כן צריכים לקרוא את הקוד — אתם, בעוד חודש, וגם חברים לצוות. קוד נקי הוא קוד שמסביר את עצמו: מישוהו שלא כתב אותו יכול להבין **מה הוא עושה**, בלי להריץ אותו ובלי לנחש.

טיפ. כלל אצבע מוכר בתעשייה: "קוד נכתב פעם אחת, ונקרא הרבה פעמים". השקעה בשם ברור עכשיו חוסכת בלבול מאוחר יותר.

אותו קוד, שני ניסוחים

איזו גרסה מבינים מהר יותר?

שני קטעי קוד שמחשבים בדיוק את אותו הדבר:

```
# גרסה א'
x = 85
y = 3
z = x * y

# גרסה ב'
price = 85
quantity = 3
total = price * quantity
```

שתי הגרסאות מדפיסות:

255

אותו פלט, הבודל ענק

אותה תוצאה, קריאות שונה לגמרי

שתי הגרסאות בשקופית הקודמת מחזירות בדיוק את אותו הערך — 255. אבל רק גרסה ב' אומרת לנו **מה זה בכלל מייצג**: מחיר, כמות וסכום כולל.

price, quantity, total

הקוד "מספר סיפור" בעצמו, בלי הסבר נוסף

x, y, z

עובד — אבל אחרי שבוע גם אתם לא תזכרו למה זה מיועד

עקרון מרכזי

עקרונות מתן שמות טובים

- השם צריך לתאר **מה המשתנה מייצג**, לא רק "מה יש בו טכנית"
- עדיף שם **קצת ארוך וברור** על פני שם קצר ומעורפל
- שם עקבי לאורך כל התוכנית — לא לקרוא לאותו דבר בשני שמות שונים
- **משתנה בודד** (למשל מונה בלולאה קצרה) יכול להסתפק בשם קצר כמו `i` — אבל זה חריג, לא ברירת מחדל

דוגמה חיה

מעורפל לברור

num_students
ברור מיד — מספר התלמידים

n
מספר של מה? תלמידים? מוצרים? ימים?

discount_percent
אין מקום לפרשנות

d
תאריך? מרחק? (distance)? הנחה? (discount)

תרגול מידי

שפרו את השמות

לכל אחד מהמשתנים הבאים, הציעו שם ברור יותר:

1. [קל] משתנה בשם **t** ששומר את הטמפרטורה במעלות צלזיוס — הציעו שם ברור.
2. [קל] משתנה בשם **p** ששומר את המחיר של מוצר לפני מע"מ — הציעו שם ברור.
3. [קל] משתנה בשם **res** ששומר את תוצאת הבדיקה אם תלמיד עבר מבחן — (True/False) הציעו שם ברור.

עכשיו לכלים

מה מותר בשם משתנה?

מעבר לטעם הטוב, ל Python יש גם **כללי תחביר נוקשים** — שם משתנה שלא עומד בהם פשוט לא יעבוד.

- מותר להשתמש ב**אותיות** (a-z, A-Z), **ספרות** (0-9) וב**קו תחתון** _
- השם **חייב להתחיל** באות או בקו תחתון — לעולם לא בספרה
- אסור **רווחים** או תווים מיוחדים כמו -, !, @ בתוך השם
- Python **רגישה לאותיות גדולות/קטנות** — Score(case-sensitive) ו-score הם שני משתנים שונים!

דוגמה · שמות חוקיים

שמות תקינים בפייתון

כל השמות האלה חוקיים לגמרי:

```
age = 18
student_name = "נועה"
_temp = 0
score2 = 95
totalPrice = 150 # עוד רגע – Python חוקי, אבל לא המוסכמה של
```

דוגמה · שמות לא חוקיים

שמות שלא יעבדו

כל השורות האלה יגרמו לשגיאת תחביר:

```
2nd_place = 1 # מתחיל בספרה – אסור
student name = "דן" # יש רווח בשם – אסור
total-price = 200 # מקף מתפרש כחיסור – אסור
```

שגיאה:

```
SyntaxError: invalid syntax
```

שימו לב

Python רגישה לאותיות

Python מבדילה בין אותיות גדולות לקטנות — אלה שני משתנים **נפרדים לגמרי**, גם אם הם "נראים" דומים:

Score

משתנה שונה לגמרי, לא אותו אחד!

score

משתנה אחד

שימו לב. זו טעות נפוצה מאוד למתחילים: להגדיר Total ואז לנסות להדפיס total — ותקבלו שגיאה, כי מבחינת Python הם לא אותו דבר.

דוגמה: הרגישות לרישיות

מלכודת נפוצה

מה קורה כאן?

```
Total = 100
print(total)
```

שגיאה:

```
NameError: name 'total' is not defined
```

תרגול: חוקיות שמות

חוקי או לא?

עברו על הרשימה וסמנו לעצמכם מה חוקי ומה לא, ולמה:

1. [קל] בדקו את השמות הבאים וקבעו אם כל אחד חוקי בפייתון: `_score`, `user-name`, `3rd_try`, `my_age`: **Grade1**.

2. [קל] נסו בעצמכם ב: **Python**-הגדירו `Age = 20` ואז נסו להדפיס `age` — מה קורה, ולמה?

3. [בינוני] תקנו שלושה שמות לא חוקיים לבחירתכם מהתרגיל הראשון כך שיהיו חוקיים, בלי לפגוע במשמעות שלהם.

המוסכמה של Python

snake_case — איך פייתון קוראת לדברים

כשבשם משתנה יש כמה מילים, איך מפרידים ביניהן? בפייתון המוסכמה הרשמית ונגסה יללכ, **PEP 8**-מ קלח) (הפשה לש סיימשרה היא **snake_case**: אותיות קטנות בלבד, עם קו תחתון בין מילים.

totalPrice

— camelCase נפוץ ב, JavaScript/Java-לא בפייתון

total_price

— snake_case כותבים בPython-

דוגמה

snake_case בפעולה

כמה משתנים בשם רב-מילים, כתובים כמו שנהוג ב: Python-

```
first_name = "מיכל"
total_score = 95
is_passing_grade = True

print(first_name, total_score)
```

פלט:

מיכל 95

טיפ נוסף

אורך סביר, לא קיצוני

שם משתנה טוב הוא **איזון**: לא כל כך קצר שהוא מעורפל, ולא כל כך ארוך שהוא מעמיס על הקריאה.

the_total_number_of_items_in_cart

ארוך מדי — קשה לקרוא

num_items

בדיוק במידה

n

קצר מדי — לא ברור

תרגול snake_case ·

תרגלו שמות נכונים

תרגלו כתיבה במוסכמת: snake_case

1. [קל] כתבו מחדש בסגנון: num0fStudents, TotalAmount, userNameSnake_case.
2. [בינוני] צרו תוכנית קטנה עם שלושה משתנים בשמות snake_case: שם מוצר, מחיר וכמות במלאי — הדפיסו את כולם.

נושא חדש

מה זן מילים שמורות?

מילים שמורות (reserved words / keywords) הן מילים שלפייתון עצמה יש להן **משמעות מיוחדת** בשפה — כמו for, if או True.

בדיוק בגלל זה **אסור** להשתמש בהן כשם משתנה Python — תזרוק שגיאת תחביר, כי היא לא יכולה להבין אם התכוונתם למילה השמורה או לשם משתנה חדש.

רשימה חלקית

כמה מילים שמורות נפוצות

import • in • not • or • and • None • False • True • return • def • continue • break • while • for • else • elif • if • class •

טיפ. לא צריך לשנן את הרשימה המלאה עכשיו — נכיר את רוב המילים האלה בהדרגה ביחידות הבאות. חלק מהן, כמו True ו-False, כבר פגשתם.

דוגמה · שגיאה

מה קורה כשמנסים בכל זאת?

ניסיון להשתמש במילה שמורה כשם משתנה:

```
for = 5
```

שגיאה:

```
SyntaxError: invalid syntax
```

טיפ מעשי

איך מזהים מילה שמורה?

לא צריך לנחש בעל-פה — יש דרך פשוטה לזהות מילה שמורה: כמעט בכל עורך קוד (Thonny, VS Code, PyCharm) מילים שמורות מסומנות **בצבע שונה** משאר הטקסט, בדרך כלל צבע בולט כמו סגול או כתום.

שימו לב. אם ניסיתם לקרוא למשתנה בשם מסוים, ופייתון "מתעקשת" לזרוק שגיאת תחביר בלי סיבה ברורה — בדקו קודם אם בטעות בחרתם מילה שמורה.

מותר או אסור?

בדקו כל שם — האם מותר להשתמש בו כשם משתנה בפייתון?

1. [קל] בדקו אילו מהשמות הבאים חוקיים לשימוש כמשתנה: `return`, `score_true`, `True`, `grade`, `class`.

2. [קל] נסו בעצמכם ב-Python להריץ `True = 5` — מה קורה, ולמה זה שונה מלנסות `my_true = 5`?

3. [בינוני] הציעו שם משתנה חוקי וברור במקום כל אחד מהשמות האסורים שמצאתם בתרגיל הראשון.

נושא חדש

למה כותבים הערות בקוד?

הערה (comment) היא שורת טקסט בתוך הקוד שפייתון **מתעלמת ממנה לגמרי** בזמן הרצה — היא נועדה רק לבני אדם שקוראים את הקוד.

- מסבירה **למה** קטע קוד קיים, לא רק מה הוא עושה
- עוזרת לכם עצמכם לחזור לקוד ישן ולהבין אותו מהר
- מאפשרת לחברי צוות להבין קוד שכתבתם, בלי לשאול אתכם ישירות

תחביר בסיסי

הערות שורה בודדת עם

כל מה שמופיע אחרי הסימן # ועד סוף השורה נחשב הערה, ופייתון פשוט מדלגת עליו.

בסוף שורת קוד

הערה קצרה שמסבירה את אותה שורה

שורה נפרדת

הערה על השורה שאחריה

דוגמה

הערות בפעולה

שני סגנונות שימוש בהערה — לפני שורה, ובסוף שורה:

```
# מחשבים את המחיר הכולל אחרי הנחה קבועה
price = 200
discount = 30 # הנחה קבועה בשקלים
total = price - discount

print(total)
```

פלט:

170

כמה שורות

הערות מרובות שורות

כדי לכתוב הערה שמשתרעת על כמה שורות, פשוט מתחילים כל שורה ב-# משלה. אין תחביר מיוחד לבלוק הערות ארוך — כל שורה מקבלת סימן משלה.

טיפ. יש בפייתון גם מחרוזות תלת-מרכאות ('''...''') (שלעיתים משמשת כהערת בלוק — נכיר אותה בהרחבה כשנלמד תיעוד פונקציות ביחידה 11)

דוגמה · הערת בלוק

הסבר מקדים לקטע קוד

כמה שורות הערה שמסבירות מה עומד לקרות בהמשך:

```
# התוכנית מחשבת את הציון הסופי של תלמיד.
# הציון הוא ממוצע משוקלל: 70% מבחן, 30% תרגילים.
exam = 85
homework = 95
final_grade = exam * 0.7 + homework * 0.3

print(final_grade)
```

פלט:

88.0

לא כל הערה טובה

הערות טובות מול הערות מיותרות

הערה טובה מסבירה **למה** — (why) לא **מה** (what), כי את ה"מה" כבר רואים בשורת הקוד עצמה.

#פיצוי על אינדקס שמתחיל מ-0

שימושית — מסבירה למה מוסיפים 1

#מוסיפים 1 ל-x

מיותרת — רואים את זה ישר בקוד $x = x + 1$

שימו לב. גם עודף הערות מזיק: אם כל שורה מוערת, קשה למצוא את ההערות שבאמת חשובות. הערה נחוצה במקום שבו הקוד לבדו לא מספיק ברור.

תרגול · חלק א'

משימות — רמה קלה

תרגול בסיסי בכתיבת הערות ובבחירת שמות.

1. [קל] קחו קטע קוד עם המשתנים `a`, `b`, `c`, שמחשב שטח משולש ($c = a * b / 2$), ושנו את שמות המשתנים לשמות ברורים בסגנון `snake_case`.
2. [קל] הוסיפו הערת שורה בודדת שמסבירה מה עושה שורת קוד שמחשבת מספר ימים בשנה (`days = 365`).
3. [קל] כתבו הערת בלוק בת שתי שורות שמסבירה מטרת תוכנית שמחשבת מחיר עם מע"מ.

תרגול · חלק ב'

משימות — רמה בינונית

כאן משלבים כמה עקרונות ביחד.

1. [בינוני] קטע הקוד `n1 = 20; n2 = 5; r = n1 // n2` מחשב כמה קופסאות שלמות של 5 בקבוקים אפשר לארוז מתוך 20 בקבוקים. שנו שמות, הוסיפו הערה מסבירה, והדפיסו את התוצאה עם תווית.
2. [בינוני] בדקו את השורה `2total = 100` — הסבירו למה היא לא תעבוד, ותקנו אותה לשם חוקי.
3. [בינוני] כתבו תוכנית קטנה שמחשבת מחיר סופי אחרי הנחה באחוזים, עם שמות משתנים ברורים ב-`snake_case` והערה אחת שמסבירה את הנוסחה.

תרגול · חלק ג'

משימות — רמה מאתגרת

תרגילי שכתוב (refactoring) לקוד בעייתי.

1. [מאתגר] שכתבו מחדש את הקוד הבא: `x = 40 y = 8 z = x * y - 15` תנו לו הקשר — נניח שמדובר בחישוב שכר לפי שעות עבודה ותעריף לשעה, בניכוי מס קבוע — עם שמות ברורים והערה שמסבירה את החישוב.
2. [מאתגר] מצאו ותקנו את כל השגיאות בקטע: `for, Total Score = 1st_score + 5, 1st_score = 90: Total Score =` — הסבירו בקצרה למה כל שורה שגויה.
3. [מאתגר] כתבו תוכנית קטנה לחישוב טיפ במסעדה, עם 3 משתנים לפחות, שמות `snake_case` ברורים, ולפחות שתי הערות שמסבירות את ה"למה" (מאחורי החישוב) לא רק את ה"מה"!).

בדיקה עצמית

שאלות חזרה מהירות

בלי לגלול אחורה — כמה מהן אתם זוכרים?

1. [קל] מה שלושת סוגי התווים המותרים בשם משתנה בפייתון?
2. [קל] האם `Score` ו-`score` הם אותו משתנה? למה?

3. [בינוני] מה ההבדל בין `snake_case`, `camelCase` ואיזה מהם המוסכמה בפייתון?

4. [בינוני] למה אסור להשתמש במילה שמורה כמו `for` כשם משתנה?

5. [בינוני] מה ההבדל בין הערה טובה להערה מיותרת?

לפני שסוגרים

חמישה דברים לזכור מהיום

`snake_case`

מוסכמת השמות הרשמית של Python

אותיות, ספרות, _

מותרים בשם, אך אסור להתחיל בספרה או להשתמש ברווח

שם ברור

מתאר מה המשתנה מייצג — לא רק "עובד טכנית"

#הערה

מסבירה "למה", לא חוזרת על מה שכבר רואים בקוד

מילים שמורות

`True`, `for`, `if`, `False` — אסורות כשם משתנה

הקוד שלכם עכשיו נקי וברור

בשיעור הבא: קלט ופלט ועבודה עם מחרוזות — נלמד להקשיב למשתמש בעזרת `input()`, ולא רק לדבר אליו.